PROJECT REPORT

Learn to Build an AI-Based Traffic Signal Controller Using VLSI

Verilog RTL Design • Finite State Machine • Adaptive Timing • Pedestrian Handling

Project Focus	Adaptive traffic signal control using modular Verilog RTL
Design Tools	Verilog HDL, Icarus Verilog, GTKWave, FPGA design tools
Implementation Scope	Educational prototype; rule-based density classification

Prepared for academic learning and FPGA/VLSI prototyping.

1. Abstract

This project presents the design and simulation of an adaptive traffic signal controller using Verilog Register Transfer Level (RTL) design. The system uses a Finite State Machine (FSM) to control red, yellow, and green traffic signals. Green-light duration is adjusted according to vehicle density, while pedestrian requests are stored and handled through a dedicated crossing phase. A reusable programmable timer manages the duration of each phase. The design is divided into modular components and verified using Verilog simulation testbenches.

2. Problem Statement

Conventional traffic signal systems often operate with fixed timing intervals, regardless of actual traffic conditions. This can lead to unnecessary waiting times and inefficient traffic flow. The proposed project addresses this limitation by implementing a density-adaptive controller that provides longer green-light durations for higher traffic density and supports pedestrian crossing requests.

3. Objectives

- To design a synthesizable traffic signal controller using Verilog HDL.
- To implement an FSM for controlling traffic signal sequences.
- To adjust green-light duration according to vehicle density.
- To develop a reusable programmable timer module.
- To implement pedestrian-request storage and controlled servicing.
- To verify system functionality using simulation and waveform analysis.
- To develop a modular design suitable for FPGA and VLSI learning.

4. Block-Level Architecture

The system consists of a vehicle-density classifier, a traffic signal controller, a programmable timer, and output logic.

Vehicle Count	→	**Density Logic**	→	**Traffic FSM**	→	**Signal Outputs**

The density logic classifies incoming vehicle-count data into low, medium, or high density. Based on this classification, the FSM selects the appropriate green-light duration and controls transitions between traffic signal phases.

The programmable timer counts clock cycles and generates a `timer_expired` pulse when the selected interval is completed. The FSM uses this pulse to advance to the next phase. Pedestrian requests are latched and evaluated at the end of the red phase. During pedestrian crossing, the vehicle red signal and `pedestrian_active` output are asserted together.

5. Module Description

5.1 Density Logic (`density_logic.v`)

This module converts an 8-bit vehicle count into a two-bit density classification.

- Low density (00): 0–9 vehicles.
- Medium density (01): 10–29 vehicles.
- High density (10): 30 or more vehicles.

5.2 Traffic Timer (`traffic_timer.v`)

This module implements a programmable counter that measures the duration of each traffic signal phase. When the selected timer limit is reached, it generates a one-clock-cycle `timer_expired` pulse and resets the counter. A zero timer limit is treated as an immediate expiry.

5.3 Traffic FSM (`traffic_fsm_final.v`)

This module controls the traffic signal sequence using a finite state machine. The normal sequence is GREEN → YELLOW → RED. At the end of the red phase, the controller enters the PEDESTRIAN phase if a request is pending; otherwise, it returns to GREEN. The pedestrian phase returns to GREEN after its timer expires.

The outputs are decoded according to the current state using Moore-style output logic.

5.4 Top-Level Controller (`traffic_system_top_final.v`)

This module integrates the FSM and timer using named port connections. It accepts a density classification and a pedestrian request and generates the traffic signal outputs.

5.5 Vehicle-Count Wrapper (`traffic_system_with_count.v`)

This optional module connects the density classifier to the traffic controller. It allows the system to receive a raw vehicle count rather than a preclassified density value.

6. Timing Configuration

The following timing values are used to demonstrate the adaptive behavior of the controller.

Traffic Phase	Duration (Clock Cycles)
Green – Low Density	5
Green – Medium Density	10
Green – High Density	20
Yellow	3
Red	8
Pedestrian Crossing	6

These values are intended for simulation purposes and do not represent real-world seconds. The actual duration depends on the clock frequency and timer implementation. For example, with a 50 MHz clock, one clock cycle lasts 20 ns. Therefore, a five-cycle interval lasts 100 ns. Practical applications require appropriately scaled timing parameters and thorough safety validation.

7. Verification and Testing

The system is verified using Verilog testbenches to examine signal transitions, density classification, timer operation, and pedestrian-request handling. The following tests are performed:

- Apply reset and verify the initial RED state.
- Test low, medium, and high density inputs and observe the corresponding green-light durations.
- Apply a pedestrian request and verify that it remains pending until serviced.
- Confirm that the vehicle red signal remains active throughout the pedestrian phase.
- Inspect the generated VCD waveform for FSM transitions and timer-expiry pulses.
- Test vehicle counts of 0, 9, 10, 29, 30, and 255 to verify density-classification thresholds.

The integrated testbench checks that green and yellow outputs are not active simultaneously and that pedestrian mode implies an active red signal. These checks provide basic functional verification but do not establish complete system safety.

8. AI-Based Functionality and Current Limitations

The current RTL implementation uses a rule-based approach to classify traffic density. It applies predefined vehicle-count thresholds and adjusts the green-light duration accordingly. It does not contain a trained machine-learning model or a camera-based vehicle-detection system.

To develop a complete AI-assisted traffic controller, an external camera or sensor system can collect traffic data, while an edge-AI processor estimates the number of vehicles. The resulting vehicle count or density classification is then supplied to the FPGA-based controller.

Camera	→	**Edge-AI Vehicle Detection**	→	**FPGA Input**	→	**Density Logic / FSM / Timer**	→	**Signal Driver**

The camera, AI inference processor, and physical signal-driver circuitry are outside the scope of the current RTL implementation.

9. Applications

- Educational demonstrations of digital logic and FSM design.
- FPGA-based traffic signal control prototypes.
- Learning Verilog HDL and modular RTL development.
- Simulation and waveform analysis of adaptive control systems.
- Integration experiments involving external vehicle-detection systems.

10. Future Enhancements

- Integrate an AI-based vehicle-counting system or validated sensor interface.
- Extend the design to support multiple traffic lanes and emergency-vehicle priority.
- Implement watchdog monitoring, sensor-failure detection, and safe fallback operation.
- Introduce interlocks to prevent conflicting traffic signals.
- Add assertions and constrained-random verification to improve test coverage.
- Scale timer parameters according to the selected FPGA clock frequency.
- Synchronize asynchronous input signals to the system clock.
- Perform FPGA synthesis, placement, routing, and resource-utilization analysis.
- Evaluate timing performance and explore ASIC implementation workflows.

11. Conclusion

The project demonstrates the design of an adaptive traffic signal controller using Verilog RTL and VLSI design principles. By combining density classification, a finite state machine, programmable timing, and pedestrian-request handling, the system provides a modular foundation for traffic-control simulation and FPGA development.

Although the current design uses rule-based density classification rather than a trained AI model, it can be extended by integrating an external AI-based vehicle-detection system. The project is intended for educational and prototyping purposes and is not certified for controlling real-world traffic signals. Practical deployment would require independent fault handling, validated fail-safe behavior, and professional safety engineering.